modifications. This capability is made possible due to the JVM. When a .java file is compiled, the Java compiler generates .class files containing bytecode with the same class names present in the .java file. When we run the .class file, it undergoes several steps, collectively describing the entire JVM process.

Class Loader

The Class Loader Subsystem is responsible for three primary activities, namely loading, linking, and initialization.

Loading

Loading involves reading the ".class" file, generating the corresponding binary data, and storing it in the method area. For each ".class" file, the JVM stores information such as the fully qualified name of the loaded class and its immediate parent class, whether the ".class" file is related to a class, interface, or enumeration, and information on modifiers, variables, and methods.

Once the ".class" file has been loaded, the JVM generates an object of type Class to represent this file in the heap memory. This object is of the predefined Class type found in the java.lang package. Programmers can use these Class objects to obtain class-level information such as the class name, parent name, method and variable information, etc. To obtain a reference to this object, we can use the getClass() method of the Object class.

```
// A Java program to demonstrate working
// of a Class type object created by JVM
// to represent .class file in memory.
import java.lang.reflect.Field;
import java.lang.reflect.Method;

// Java code to demonstrate use
// of Class object created by JVM
public class Test {
      public static void main(String[] args)
      {
              Student s1 = new Student();

              // Getting hold of Class
              // object created by JVM.
              Class c1 = s1.getClass();

              // Printing type of object using c1.
```